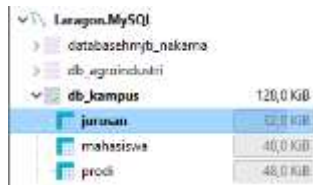
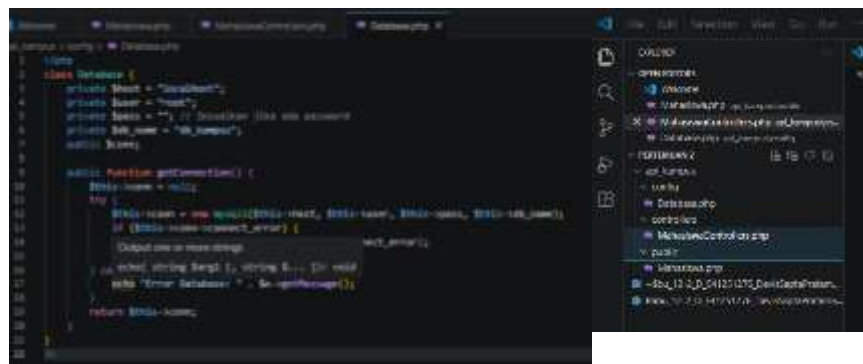


Nama : Devis Sapta Pratama
NIM : E41251276
Golongan : D

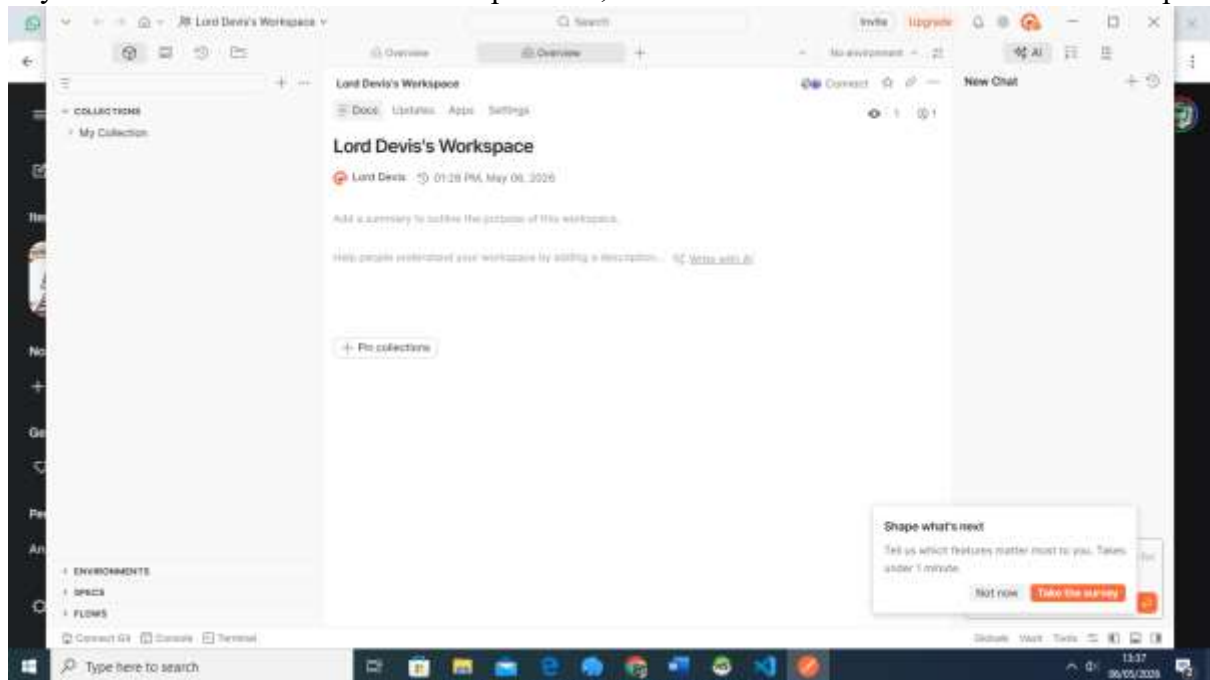


Hal pertama yang dilakukan adalah mengaktifkan laragon terlebih dahulu. Kemudian saya membuat database baru bernama db_kampus pada HeidiSQL. Kemudian saya menjalankan *query* SQL yang ada di modul (Tahap 1) untuk membuat tabel jurusan, prodi, dan mahasiswa. Saya memastikan bahwa relasi *foreign key*-nya terbentuk dengan benar, karena ini krusial untuk menjaga integritas data sistem akademik.

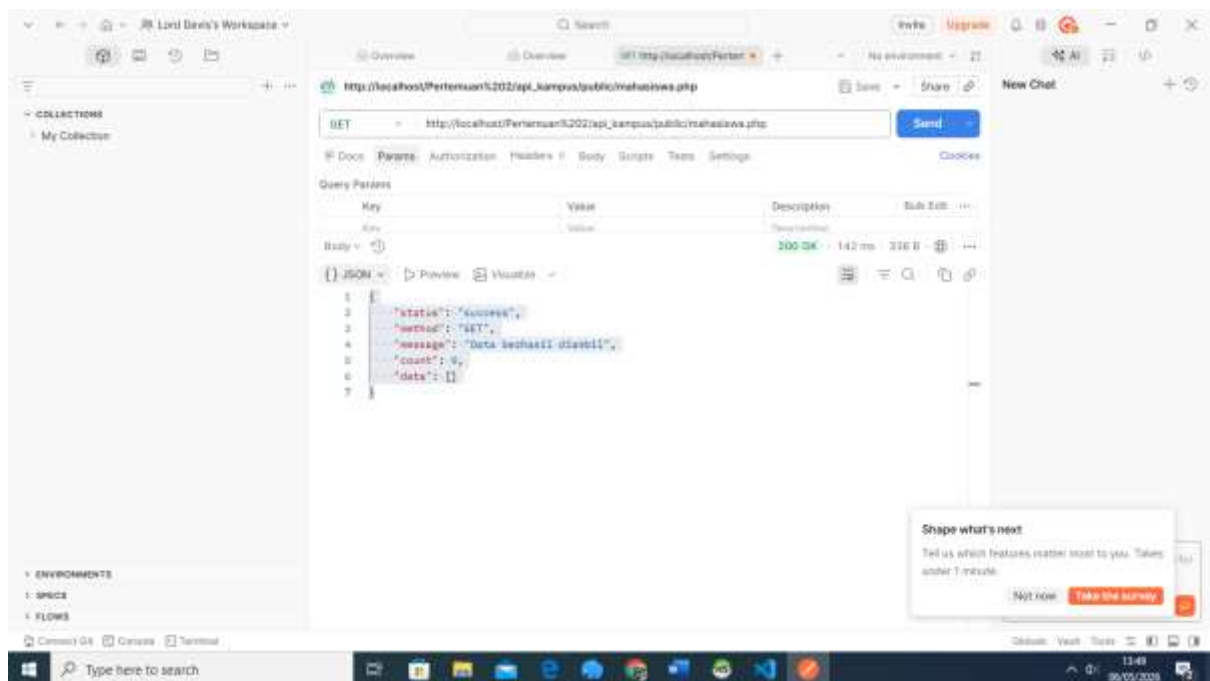
Kemudian saya membuat folder dan filenya didalam visual studio code. Kemudian didalam file database yang berada didalam berkas config, saya memasukkan kelas Database.



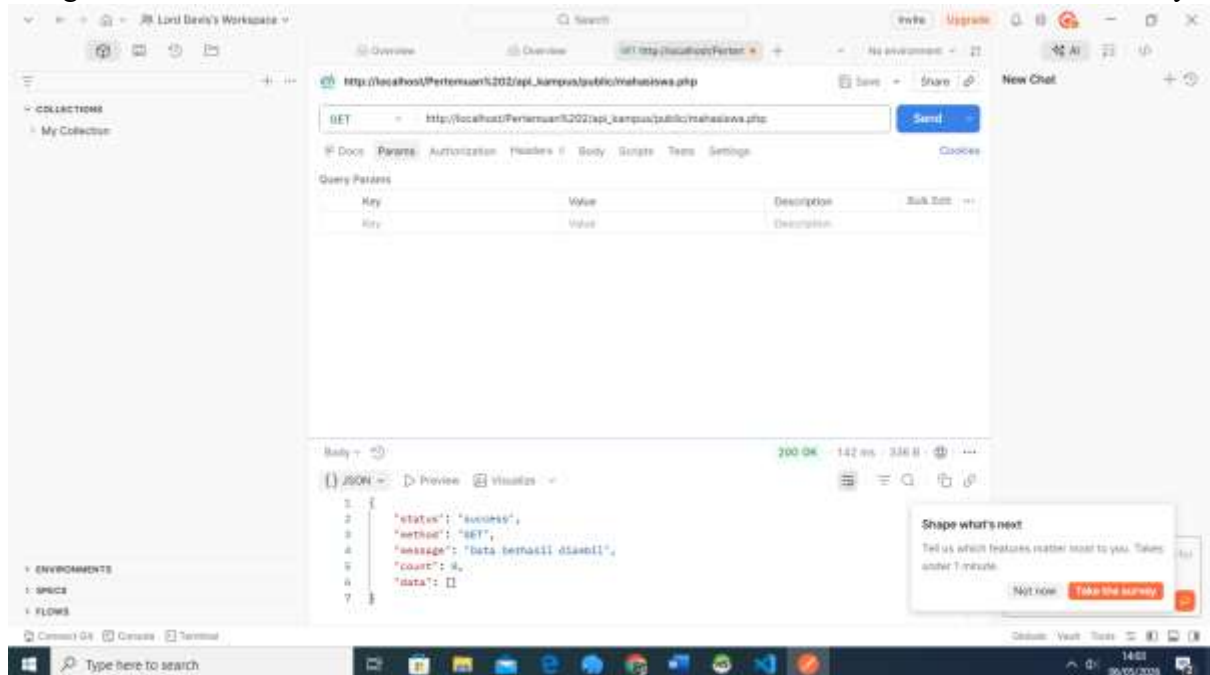
Selanjutnya saya mengisi kode didalam file MahasiswaControllernya seperti modul dibkpm. Saya mendownload postman, sesuai arahan bkpm.



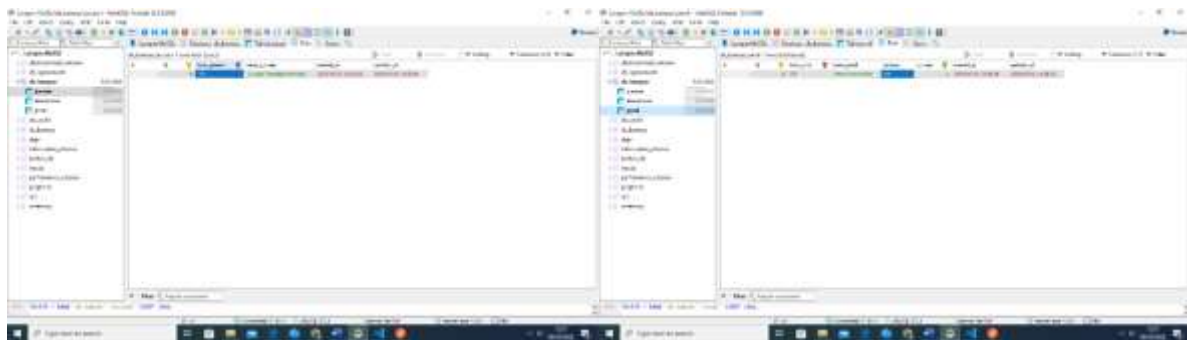
Kemudian menjadi seperti ini :



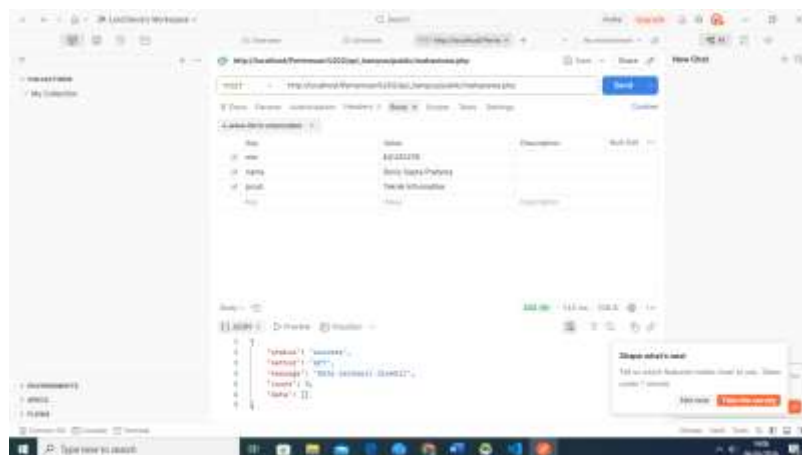
Kemudian pada tampilan utama Postman, saya mengeklik ikon "+" (tanda tambah) di deretan atas untuk membuka tab *request* baru. Saya memastikan kotak *dropdown* di sebelah kiri kolom URL sudah terpilih metode **GET**. Kemudian saya mengetikkan URL endpoint API Anda ke dalam kotak panjang tersebut. Saya menyesuaikan bagian nama_folder dengan folder *project* yang saya buat di dalam htdocs. Formatnya seperti ini: http://localhost/Pertemuan%202/api_kampus/public/mahasiswa.php. Kemudian saya mengklik tombol biru **"Send"** di sebelah kanannya.



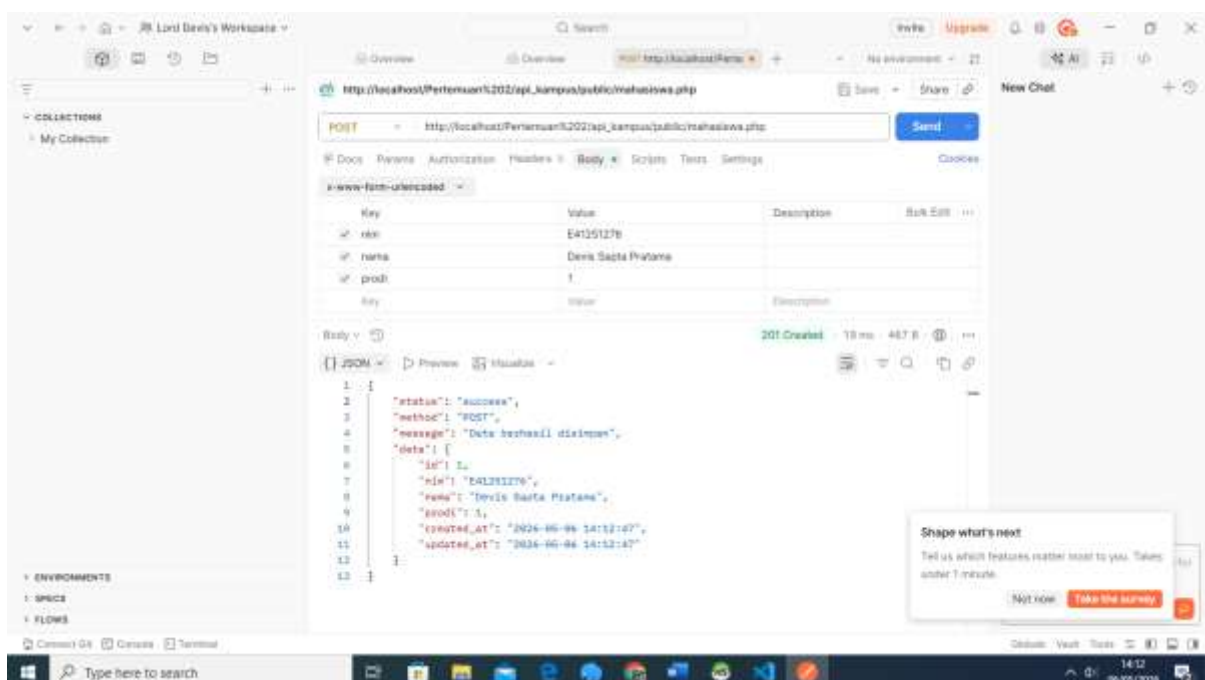
Kemudian saya akan mengujinya dengan cara mengisi didalam database jurusan dan prodi, bebas aja sih yang penting ada isinya supaya bisa dicoba.



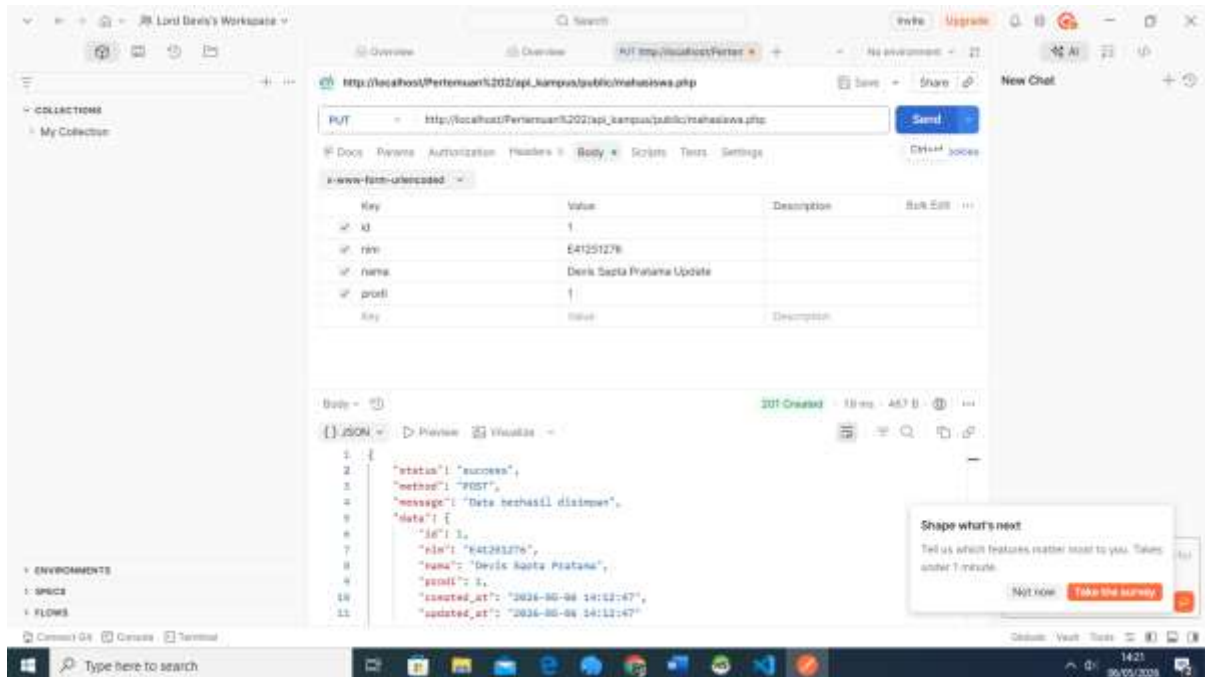
Kemudian saya mengubah metode *dropdown* di sebelah kiri URL dari **GET** menjadi **POST**. URL-nya biarkan sama. Tepatnya di bawah baris URL, saya mengklik tab **Body**. Saya memilih opsi **x-www-form-urlencoded** (ini adalah format standar yang dibaca oleh \$_POST di PHP). Kemudian dihalaman akan muncul tabel isian *Key* dan *Value*. Saya mengisinya seperti ini:



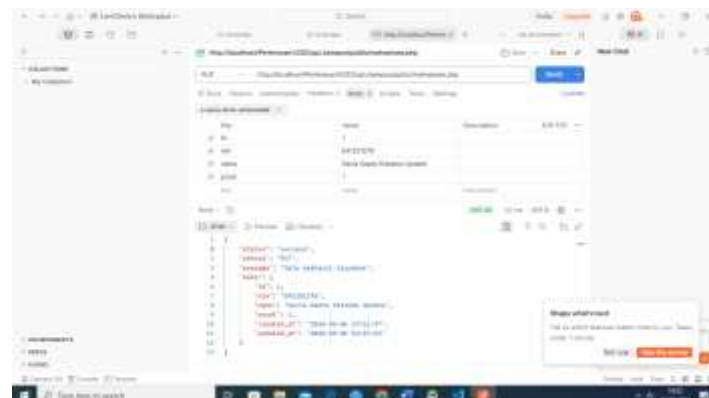
Kemudian saya mengklik tombol send berwarna biru diatas dan menjadi seperti ini :



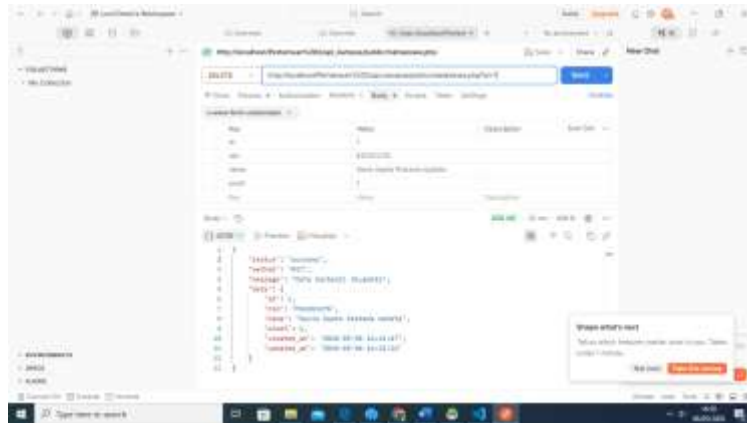
Selanjutnya adalah pengujian put, saya mengubah metode *dropdown* di Postman dari POST menjadi **PUT**. Saya membiarkan URL-nya sama. Tetap di tab **Body**, dan saya memastikan masih memilih **x-www-form-urlencoded**. Sekarang, kita butuh memasukkan ID data mana yang mau diubah. Saya menambahkan **id** di tabel pengisiannya dan menjadi seperti ini :



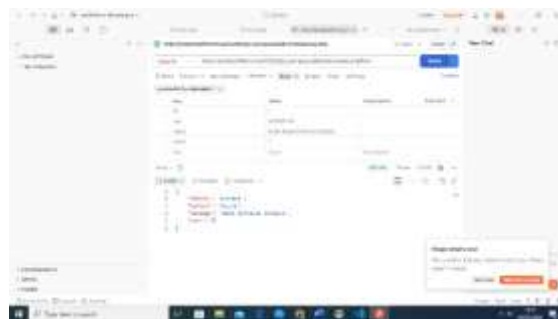
ohh, iya saya menambahkan update pada Namanya dan mengklik **Send** dan menjadi seperti ini:



Selanjutnya adalah metode delete. Nah, untuk DELETE ini ada sedikit perbedaan. Saya memperhatikan kode PHP saya untuk bagian DELETE: `$id = $_GET['id'] ?? null;` Karena menggunakan `$_GET`, berarti ID-nya harus dikirim lewat **URL**, bukan lewat *Body*. Jadi saya mengubah metode *dropdown* di Postman menjadi **DELETE**. Kemudian menghapus semua centang di tab *Body* (atau pilih *none*), karena kita tidak mengirim data lewat *body*. Kemudian saya mengubah **URL** Anda dengan menambahkan `?id=1` di bagian paling belakang. Sehingga menjadi seperti ini:



Kemudian saya mengklik **Send** dan menjadi seperti ini:



Kesimpulan

Melalui praktikum ini, implementasi arsitektur *Client-Service-API* yang terdistribusi menggunakan PHP Native OOP telah berhasil dilakukan. Seluruh operasi CRUD (Create, Read, Update, Delete) dapat dieksekusi dengan baik melalui *endpoint* API yang telah dibangun. Pengujian menggunakan aplikasi Postman membuktikan bahwa server mampu merespons berbagai *request* metode HTTP (GET, POST, PUT, DELETE) secara dinamis dan mengembalikan *output* dalam format JSON yang valid (HTTP Status Code 200 OK dan 201 Created). Pemisahan logika ke dalam *controller* dengan pengamanan *prepared statements* juga memastikan integritas dan keamanan database tetap terjaga.

Refleksi

Praktikum ini memberikan pemahaman teknis yang mendalam tentang bagaimana *backend* berinteraksi dengan *client* secara terstruktur (*microservices readiness*). Berbagai tantangan *debugging*, seperti penyesuaian *routing* direktori server lokal (localhost), penanganan struktur relasi *Foreign Key* saat melakukan *insert* data, hingga spesifikasi pengiriman parameter (via *Body* maupun URL), telah memberikan pengalaman penyelesaian masalah yang berharga. Keterampilan fundamental dalam membangun RESTful API ini akan menjadi fondasi yang sangat esensial untuk pengembangan sistem informasi maupun aplikasi manajemen database yang lebih kompleks dan berskala besar ke depannya.